

05 - I2C & UART Peripherals - Session Notes

I2C and UART Production Patterns

1. Communication Protocols — Why They Exist

Your ESP32 cannot do everything alone. It needs to talk to:

- Sensors — temperature, humidity, pressure, accelerometer
- Displays — OLED screens, LCD panels
- Other chips — GPS modules, motor controllers, audio codecs

These conversations need a common language. That language is a **communication protocol** — a set of rules about how data is sent and received.

PC Analogy

Think of different ways your PC communicates:

PC Communication Methods

USB	Fast, many devices, plug-and-play
HDMI	Video only, one direction
Ethernet	Networked, long distance
Headphone jack (3.5mm)	Simple, audio only
Bluetooth	Wireless, short range

ESP32 Communication Protocols

I2C	2 wires, many devices, slow-medium
SPI	4 wires, fast, one device at once
UART	2 wires, point to point, simple
I2S	Audio data
CAN	Automotive, long distance

In this session we focus on the two most common: **I2C** — for sensors and displays sharing a bus **UART** — for point-to-point serial communication

2. I2C — The Two Wire Protocol

I2C (Inter-Integrated Circuit — pronounced "I squared C") is a protocol that lets multiple devices share just **two wires**.

The Two Wires

Common I2C Device Addresses

Device	Address
SSD1306 OLED	0x3C or 0x3D
BMP280 sensor	0x76 or 0x77
SHT31 sensor	0x44 or 0x45
MPU6050 gyro	0x68 or 0x69
DS3231 RTC	0x68
ADS1115 ADC	0x48 to 0x4B

If two devices have the same address — they will conflict. This is why most I2C sensors have an address pin you can connect to VCC or GND to choose between two addresses.

3. I2C in ESP-IDF — Initialisation

The ESP-IDF I2C Driver

ESP-IDF v5.x and v6.x introduced a new I2C driver API. We use the new API — it is cleaner and more reliable.

Step 1 — Configure and Install I2C Master

```
#include "driver/i2c_master.h"

#define I2C_MASTER_SCL_IO    GPIO_NUM_22
#define I2C_MASTER_SDA_IO    GPIO_NUM_21
#define I2C_MASTER_FREQ_HZ    400000    // 400 kHz Fast Mode
#define I2C_MASTER_PORT      I2C_NUM_0

static i2c_master_bus_handle_t i2c_bus_handle;

static void i2c_master_init(void)
{
    i2c_master_bus_config_t bus_config = {
        .clk_source      = I2C_CLK_SRC_DEFAULT,
        .i2c_port        = I2C_MASTER_PORT,
        .scl_io_num       = I2C_MASTER_SCL_IO,
        .sda_io_num       = I2C_MASTER_SDA_IO,
        .glitch_ignore_cnt = 7,
        .flags.enable_internal_pullup = true,
    };

    ESP_ERROR_CHECK(i2c_new_master_bus(
        &bus_config,
        &i2c_bus_handle
    ));

    ESP_LOGI(TAG, "I2C master initialised on SDA=%d SCL=%d at %d Hz",
        I2C_MASTER_SDA_IO,
```

```

        I2C_MASTER_SCL_IO,
        I2C_MASTER_FREQ_HZ);
}

```

Step 2 — Add a Device to the Bus

Each device on the I2C bus needs to be registered separately:

```

static i2c_master_dev_handle_t sensor_dev_handle;

static void i2c_device_add(uint8_t device_address)
{
    i2c_device_config_t dev_config = {
        .dev_addr_length = I2C_ADDR_BIT_LEN_7,
        .device_address  = device_address,
        .scl_speed_hz    = I2C_MASTER_FREQ_HZ,
    };

    ESP_ERROR_CHECK(i2c_master_bus_add_device(
        i2c_bus_handle,
        &dev_config,
        &sensor_dev_handle
    ));
}

```

Step 3 — Read and Write

```

// Write to device
uint8_t write_buf[2] = {REG_ADDR, VALUE};
ESP_ERROR_CHECK(i2c_master_transmit(
    sensor_dev_handle,
    write_buf,
    sizeof(write_buf),
    1000 // timeout ms
));

// Read from device
uint8_t read_buf[6];
ESP_ERROR_CHECK(i2c_master_receive(
    sensor_dev_handle,
    read_buf,
    sizeof(read_buf),
    1000 // timeout ms
));

// Write then Read (most common - send register address, then read data)
uint8_t reg_addr = 0xF3;
uint8_t data[2];
ESP_ERROR_CHECK(i2c_master_transmit_receive(
    sensor_dev_handle,
    &reg_addr, 1, // write: register address
    data, sizeof(data), // read: data bytes
));

```

```
1000
));
```

4. Scanning the I2C Bus

Before writing driver code — always scan the bus to confirm your device is connected and responding.

```
#include "driver/i2c_master.h"
#include "esp_log.h"

static const char *TAG = "i2c_scan";

void i2c_scan(i2c_master_bus_handle_t bus_handle)
{
    ESP_LOGI(TAG, "Scanning I2C bus...");
    ESP_LOGI(TAG, "      0  1  2  3  4  5  6  7  8  9  A  B  C  D  E  F");

    for (uint8_t i = 0; i < 128; i += 16) {
        ESP_LOGI(TAG, "%02X: ", i);
        for (uint8_t j = 0; j < 16; j++) {
            uint8_t address = i + j;
            esp_err_t ret = i2c_master_probe(
                bus_handle,
                address,
                50 // timeout ms
            );
            if (ret == ESP_OK) {
                ESP_LOGI(TAG, "%02X ", address);
            }
        }
    }
    ESP_LOGI(TAG, "I2C scan complete");
}
```

Expected output when OLED and sensor connected:

```
I2C scan complete
Found device at address: 0x3C    <- OLED display
Found device at address: 0x76    <- BMP280 sensor
```

If nothing found — check wiring. SDA and SCL must be correct. VCC must be 3.3V — not 5V (unless your board has a voltage regulator).

5. SSD1306 OLED Display — I2C

The SSD1306 is a 128x64 pixel OLED display that communicates over I2C. It is the most common display used with ESP32 in prototypes and products.

Wiring

```
ESP32          SSD1306 OLED
GPIO 21 (SDA)  ----- SDA
GPIO 22 (SCL)  ----- SCL
3.3V           ----- VCC
GND            ----- GND
```

Using ESP-IDF Component Manager

The easiest way to drive the SSD1306 is using the official Espressif component from the IDF Component Registry.

Add to your project `main/idf_component.yml` :

```
dependencies:
  espressif/esp_lcd_ssd1306: "^1.0.0"
  idf: ">=5.0.0"
```

Then run:

```
idf.py build
# This downloads and installs the component automatically
```

OLED Initialisation Code

```
#include "esp_lcd_panel_io.h"
#include "esp_lcd_panel_vendor.h"
#include "esp_lcd_panel_ops.h"
#include "driver/i2c_master.h"
#include "esp_log.h"

static const char *TAG = "oled";

#define OLED_I2C_ADDR      0x3C
#define OLED_SCL_PIN      GPIO_NUM_22
#define OLED_SDA_PIN      GPIO_NUM_21
#define OLED_RESET_PIN    -1           // Not connected
#define OLED_H_RES        128
#define OLED_V_RES        64

static esp_lcd_panel_handle_t panel_handle = NULL;
static i2c_master_bus_handle_t i2c_bus    = NULL;

static void oled_init(void)
{
    /* Step 1 - Initialise I2C bus */
    i2c_master_bus_config_t bus_config = {
        .clk_source      = I2C_CLK_SRC_DEFAULT,
        .i2c_port        = I2C_NUM_0,
        .scl_io_num      = OLED_SCL_PIN,
        .sda_io_num      = OLED_SDA_PIN,
```

```

        .glitch_ignore_cnt = 7,
        .flags.enable_internal_pullup = true,
    };
    ESP_ERROR_CHECK(i2c_new_master_bus(&bus_config, &i2c_bus));

    /* Step 2 – Create I2C IO handle for LCD panel */
    esp_lcd_panel_io_handle_t io_handle = NULL;
    esp_lcd_panel_io_i2c_config_t io_config = {
        .dev_addr          = OLED_I2C_ADDR,
        .control_phase_bytes = 1,
        .dc_bit_offset      = 6,
        .lcd_cmd_bits       = 8,
        .lcd_param_bits     = 8,
        .on_color_trans_done = NULL,
        .user_ctx           = NULL,
        .scl_speed_hz       = 400000,
        .flags.dc_low_on_data = false,
    };
    ESP_ERROR_CHECK(esp_lcd_new_panel_io_i2c(
        i2c_bus,
        &io_config,
        &io_handle
    ));

    /* Step 3 – Initialise SSD1306 panel */
    esp_lcd_panel_dev_config_t panel_config = {
        .bits_per_pixel = 1,
        .reset_gpio_num = OLED_RESET_PIN,
    };
    esp_lcd_ssd1306_config_t ssd1306_config = {
        .height = OLED_V_RES,
    };
    panel_config.vendor_config = &ssd1306_config;

    ESP_ERROR_CHECK(esp_lcd_new_panel_ssd1306(
        io_handle,
        &panel_config,
        &panel_handle
    ));

    /* Step 4 – Reset and initialise the display */
    ESP_ERROR_CHECK(esp_lcd_panel_reset(panel_handle));
    ESP_ERROR_CHECK(esp_lcd_panel_init(panel_handle));
    ESP_ERROR_CHECK(esp_lcd_panel_disp_on_off(panel_handle, true));

    ESP_LOGI(TAG, "OLED initialised – %dx%d", OLED_H_RES, OLED_V_RES);
}

```

Drawing to the OLED

The SSD1306 works at pixel level. For text rendering we use LVGL or a simple font library. Here is the raw pixel approach using a frame buffer:

```

#include <string.h>

/* Frame buffer — 128x64 pixels, 1 bit per pixel
   Total size: 128 * 64 / 8 = 1024 bytes */
static uint8_t oled_framebuf[OLED_H_RES * OLED_V_RES / 8];

/* Clear the frame buffer */
static void oled_clear(void)
{
    memset(oled_framebuf, 0x00, sizeof(oled_framebuf));
}

/* Set a single pixel */
static void oled_set_pixel(int x, int y, int color)
{
    if (x < 0 || x >= OLED_H_RES) return;
    if (y < 0 || y >= OLED_V_RES) return;

    int byte_idx = (y / 8) * OLED_H_RES + x;
    int bit_idx = y % 8;

    if (color) {
        oled_framebuf[byte_idx] |= (1 << bit_idx);
    } else {
        oled_framebuf[byte_idx] &= ~(1 << bit_idx);
    }
}

/* Push frame buffer to display */
static void oled_flush(void)
{
    esp_lcd_panel_draw_bitmap(
        panel_handle,
        0, 0,
        OLED_H_RES, OLED_V_RES,
        oled_framebuf
    );
}

/* Draw a filled rectangle */
static void oled_fill_rect(int x, int y, int w, int h, int color)
{
    for (int row = y; row < y + h; row++) {
        for (int col = x; col < x + w; col++) {
            oled_set_pixel(col, row, color);
        }
    }
}

```

Simple Text on OLED Using LVGL

For proper text rendering — use LVGL which is also available via the IDF Component Registry:


```
# idf_component.yml
dependencies:
  lvgl/lvgl: "^8.3.0"
  espressif/esp_lvgl_port: "^1.4.0"
  espressif/esp_lcd_ssd1306: "^1.0.0"
```

```
#include "lvgl.h"
#include "esp_lvgl_port.h"

static lv_display_t *lvgl_disp = NULL;

static void lvgl_oled_init(void)
{
    /* Initialise LVGL port */
    lvgl_port_cfg_t lvgl_cfg = {
        .task_priority    = 4,
        .task_stack       = 4096,
        .task_affinity    = -1,
        .task_max_sleep_ms = 500,
        .timer_period_ms  = 5,
    };
    ESP_ERROR_CHECK(lvgl_port_init(&lvgl_cfg));

    /* Add OLED display to LVGL */
    lvgl_port_display_cfg_t disp_cfg = {
        .io_handle        = io_handle,      // from oled_init()
        .panel_handle     = panel_handle,   // from oled_init()
        .buffer_size      = OLED_H_RES * OLED_V_RES / 8,
        .double_buffer    = false,
        .hres             = OLED_H_RES,
        .vres             = OLED_V_RES,
        .monochrome       = true,
        .rotation = {
            .swap_xy      = false,
            .mirror_x     = false,
            .mirror_y     = false,
        },
        .flags = {
            .buff_spiram  = false,
        },
    };
    lvgl_disp = lvgl_port_add_disp(&disp_cfg);
}

static void oled_show_text(const char *line1, const char *line2)
{
    lvgl_port_lock(0);

    lv_obj_t *screen = lv_display_get_screen_active(lvgl_disp);
    lv_obj_clean(screen);

    /* Line 1 */
    lv_obj_t *label1 = lv_label_create(screen);
    lv_label_set_text(label1, line1);
}
```

```

lv_obj_align(label1, LV_ALIGN_TOP_MID, 0, 4);

/* Line 2 */
if (line2 != NULL) {
    lv_obj_t *label2 = lv_label_create(screen);
    lv_label_set_text(label2, line2);
    lv_obj_align(label2, LV_ALIGN_TOP_MID, 0, 24);
}

lvgl_port_unlock();
}

```

Full OLED Demo — Show Sensor Data

```

void app_main(void)
{
    oled_init();
    lvgl_oled_init();

    char line1[32];
    char line2[32];
    float temperature = 28.5f;
    float humidity    = 65.0f;

    while (1) {
        snprintf(line1, sizeof(line1), "Temp: %.1f C", temperature);
        snprintf(line2, sizeof(line2), "Humi: %.1f %%", humidity);

        oled_show_text(line1, line2);

        ESP_LOGI(TAG, "Display updated: %s | %s", line1, line2);

        vTaskDelay(pdMS_TO_TICKS(2000));
    }
}

```

Expected result on screen:

```

+-----+
|      Temp: 28.5 C      |
|      Humi: 65.0 %     |
|                       |
|                       |
|                       |
+-----+

```

6. Reading a Sensor over I2C — BMP280

The BMP280 reads temperature and atmospheric pressure over I2C.

Why BMP280?

Property	Value
Protocol	I2C or SPI
Temperature range	-40 to 85°C
Pressure range	300 to 1100 hPa
I2C address	0x76 or 0x77
Supply voltage	3.3V

Using the ESP-IDF BMP280 Component

```
# idf_component.yml
dependencies:
  espressif/bmp280: "^1.0.0"
```

```
#include "bmp280.h"

static bmp280_t bmp280_dev;

static void bmp280_sensor_init(i2c_master_bus_handle_t bus)
{
    bmp280_params_t params;
    bmp280_init_default_params(&params);

    ESP_ERROR_CHECK(bmp280_init_desc(
        &bmp280_dev,
        BMP280_I2C_ADDRESS_0, // 0x76
        I2C_NUM_0,
        I2C_MASTER_SDA_IO,
        I2C_MASTER_SCL_IO
    ));

    ESP_ERROR_CHECK(bmp280_init(&bmp280_dev, &params));

    bool is_bmp280 = bmp280_dev.id == BMP280_CHIP_ID;
    ESP_LOGI(TAG, "BMP280 found: %s", is_bmp280 ? "YES" : "NO");
}

static void bmp280_read_task(void *arg)
{
    float temperature, pressure, humidity;

    while (1) {
        if (bmp280_read_float(
            &bmp280_dev,
            &temperature,
            &pressure,
            &humidity) == ESP_OK) {

            ESP_LOGI(TAG, "Temperature: %.2f C", temperature);
            ESP_LOGI(TAG, "Pressure: %.2f hPa", pressure / 100.0f);
        }
    }
}
```

```

        /* Update OLED */
        char line1[32], line2[32];
        snprintf(line1, sizeof(line1), "%.1f C", temperature);
        snprintf(line2, sizeof(line2), "%.0f hPa", pressure / 100.0f);
        oled_show_text(line1, line2);
    }
    vTaskDelay(pdMS_TO_TICKS(2000));
}
}

```

7. UART — Serial Communication

UART (Universal Asynchronous Receiver Transmitter) is the oldest and simplest communication protocol in embedded systems.

What UART Is

Unlike I2C which uses a clock wire — UART is **asynchronous**. Both sides agree on a speed (baud rate) in advance. No clock wire needed. Just TX (transmit) and RX (receive).

PC Analogy

UART is exactly like the serial port (COM port) on old PCs:

```

Your PC (1990s)          Modem
COM1 port ----- TX ----> RX
                      ----- RX <----- TX
                      ----- GND ----- GND

```

Baud rate agreed in advance: 9600 bps
No clock wire — both sides count timing internally

USB-to-serial adapters (CP2102, CH340) still do UART under the hood. When you flash your ESP32 — you are using UART. When you see the serial monitor — you are reading UART output.

UART vs I2C

Property	I2C	UART
Wires	2 (SDA + SCL)	2 (TX + RX)
Clock	Yes — master controls	No — agreed baud rate
Multiple devices	Yes — up to 127	No — point to point
Speed	Up to 1 MHz	Up to 5 Mbps
Distance	Short — same board	Longer — metres
Common use	Sensors, displays	GPS, GSM, debug
Protocol overhead	Higher	Minimal

UART Baud Rate Reference

Baud Rate	Speed	Common Use
9600	Very slow	Legacy devices
115200	Standard	Most sensors, debug
230400	Fast	High data rate
921600	Very fast	Firmware flashing
1000000	1 Mbps	High performance

8. UART in ESP-IDF

Initialisation

```
#include "driver/uart.h"
#include "esp_log.h"

static const char *TAG = "uart";

#define UART_PORT          UART_NUM_1
#define UART_TX_PIN        GPIO_NUM_17
#define UART_RX_PIN        GPIO_NUM_16
#define UART_BAUD           115200
#define UART_BUF_SIZE      1024

static void uart_init(void)
{
    uart_config_t uart_config = {
        .baud_rate    = UART_BAUD,
        .data_bits    = UART_DATA_8_BITS,
        .parity        = UART_PARITY_DISABLE,
        .stop_bits    = UART_STOP_BITS_1,
        .flow_ctrl     = UART_HW_FLOWCTRL_DISABLE,
        .source_clk    = UART_SCLK_DEFAULT,
    };

    /* Configure UART parameters */
    ESP_ERROR_CHECK(uart_param_config(UART_PORT, &uart_config));

    /* Set UART pins */
    ESP_ERROR_CHECK(uart_set_pin(
        UART_PORT,
        UART_TX_PIN,    // TX
        UART_RX_PIN,    // RX
        UART_PIN_NO_CHANGE, // RTS - not used
        UART_PIN_NO_CHANGE // CTS - not used
    ));
}
```

```

/* Install UART driver with RX buffer */
ESP_ERROR_CHECK(uart_driver_install(
    UART_PORT,
    UART_BUF_SIZE, // RX buffer size
    0,             // TX buffer size (0 = no buffer)
    0,             // Queue size (0 = no event queue)
    NULL,          // Queue handle
    0              // Interrupt flags
));

ESP_LOGI(TAG, "UART%d initialised at %d baud on TX=%d RX=%d",
    UART_PORT, UART_BAUD, UART_TX_PIN, UART_RX_PIN);
}

```

UART Transmit

```

static void uart_send_string(const char *str)
{
    uart_write_bytes(UART_PORT, str, strlen(str));
}

static void uart_send_formatted(const char *fmt, ...)
{
    char buf[256];
    va_list args;
    va_start(args, fmt);
    vsnprintf(buf, sizeof(buf), fmt, args);
    va_end(args);
    uart_write_bytes(UART_PORT, buf, strlen(buf));
}

```

UART Receive — Event Driven

```

#include "driver/uart.h"
#include "freertos/queue.h"

#define UART_QUEUE_SIZE 20

static QueueHandle_t uart_queue;

static void uart_init_with_events(void)
{
    uart_config_t uart_config = {
        .baud_rate    = UART_BAUD,
        .data_bits    = UART_DATA_8_BITS,
        .parity        = UART_PARITY_DISABLE,
        .stop_bits    = UART_STOP_BITS_1,
        .flow_ctrl     = UART_HW_FLOWCTRL_DISABLE,
        .source_clk    = UART_SCLK_DEFAULT,
    };

    ESP_ERROR_CHECK(uart_param_config(UART_PORT, &uart_config));
    ESP_ERROR_CHECK(uart_set_pin(

```

```

    UART_PORT, UART_TX_PIN, UART_RX_PIN,
    UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE
));

/* Install driver WITH event queue */
ESP_ERROR_CHECK(uart_driver_install(
    UART_PORT,
    UART_BUF_SIZE,
    0,
    UART_QUEUE_SIZE, // Event queue depth
    &uart_queue,      // Queue handle
    0
));
}

/* Task that handles UART events */
static void uart_event_task(void *arg)
{
    uart_event_t event;
    uint8_t      rx_buf[UART_BUF_SIZE];

    while (1) {
        /* Block until UART event arrives */
        if (xQueueReceive(uart_queue, &event, portMAX_DELAY)) {

            switch (event.type) {

                case UART_DATA:
                    /* Data received */
                    int len = uart_read_bytes(
                        UART_PORT,
                        rx_buf,
                        event.size,
                        100 / portTICK_PERIOD_MS
                    );
                    rx_buf[len] = '\0';
                    ESP_LOGI(TAG, "Received %d bytes: %s", len, rx_buf);

                    /* Echo back */
                    uart_write_bytes(UART_PORT, (char *)rx_buf, len);
                    break;

                case UART_FIFO_OVF:
                    ESP_LOGW(TAG, "UART FIFO overflow - data lost");
                    uart_flush_input(UART_PORT);
                    xQueueReset(uart_queue);
                    break;

                case UART_BUFFER_FULL:
                    ESP_LOGW(TAG, "UART buffer full");
                    uart_flush_input(UART_PORT);
                    xQueueReset(uart_queue);
                    break;

                case UART_BREAK:

```

```

        ESP_LOGI(TAG, "UART break signal");
        break;

    default:
        ESP_LOGI(TAG, "UART event type: %d", event.type);
        break;
    }
}
}
vTaskDelete(NULL);
}

```

9. Combining I2C and UART — Full Example

This is the pattern you will use in Project 1. Read sensor over I2C, display on OLED over I2C, and send data over UART to a PC or another device.

```

void app_main(void)
{
    ESP_LOGI(TAG, "=== Sensor Node - I2C + UART Demo ===");

    /* Initialise I2C bus */
    i2c_master_init();

    /* Scan to confirm devices are present */
    i2c_scan(i2c_bus_handle);

    /* Initialise OLED */
    oled_init();
    lvgl_oled_init();

    /* Initialise BMP280 sensor */
    bmp280_sensor_init(i2c_bus_handle);

    /* Initialise UART */
    uart_init_with_events();

    /* Start UART event task */
    xTaskCreate(uart_event_task, "uart_task", 4096, NULL, 12, NULL);

    /* Start sensor + display task */
    xTaskCreate(bmp280_read_task, "sensor_task", 4096, NULL, 10, NULL);

    ESP_LOGI(TAG, "All systems running");
}

```

10. I2C and UART — Choosing the Right Protocol

Use this table to decide which protocol to use for a given sensor or module:

Use I2C when	Use UART when
Sensor has SDA/SCL pins	Module has TX/RX pins
Multiple devices on same bus	Point-to-point only
Short distance — same PCB	Longer distance needed
Low to medium data rate	Streaming data (GPS, GSM)
OLED displays, RTC modules	GPS modules, GSM modems
Gyroscopes, accelerometers	Barcode scanners
ADC chips, DAC chips	Bluetooth serial modules

Summary Table

Concept	I2C	UART
Wires needed	2 (SDA + SCL)	2 (TX + RX)
Clock	Master controlled	Agreed baud rate
Devices on bus	Up to 127	1 (point to point)
Max speed	1 MHz	5 Mbps
Address needed	Yes — 7 bit	No
ESP-IDF init	i2c_master_bus_config_t	uart_config_t
Send data	i2c_master_transmit()	uart_write_bytes()
Receive data	i2c_master_receive()	uart_read_bytes()
Event driven	i2c_master_transmit_receive()	uart_driver_install + queue
Common sensors	BMP280, SSD1306, MPU6050	GPS, GSM, HC-05 BT
